

Lab 0. QEMU Simulator and Raspberry Pi Setup

Revised edition — package names updated for current Ubuntu releases

We set up a simulation environment with QEMU to run Raspberry Pi code on your PC. The following instructions are for Windows 11. Note that SSH to Linux machines in the lab does not work, since you need root privileges to install QEMU on the machine.

Step 1. Set up QEMU

1.1 Enable the Windows features

From the Search box, start "Turn Windows features on or off" and enable:

- Windows Subsystem for Linux (required)
- Virtual Machine Platform (required)
- Hyper-V (optional for this lab — see the note below)

Note: This lab emulates an ARM machine on an x86 PC, which does not use hardware virtualization at all. Hyper-V is therefore not required. If you are running Windows 11 Home you will not see a Hyper-V entry in this list; that is expected, and WSL 2 still works.

1.2 Install WSL

From Windows PowerShell, run:

```
wsl --install  
wsl --set-default-version 2
```

When starting WSL, choose "run as administrator". If this creates problems with Copy and Paste, then do not run as administrator, but use "sudo" to become administrator later.

1.3 Install QEMU inside WSL

From WSL, run:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y qemu-system-arm qemu-utils
```

Verify the installation:

```
qemu-system-aarch64 --version
```

Note: Emulating a Raspberry Pi on an x86 PC is cross-architecture emulation. QEMU translates ARM instructions in software using its TCG engine, so KVM, Hyper-V and /dev/kvm play no role here. The emulated Pi will boot noticeably more slowly than real hardware — this is normal, and the first boot can take several minutes.

1.4 SKIP THIS STEP if you also want to run x86 virtual machines (optional)

Older versions of this handout, and the video above, tell you to run:

```
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils -y
```

On current Ubuntu releases this command fails, because qemu-kvm is now a virtual package with no installable candidate. KVM support has not been removed — it is provided by qemu-system-x86. None of these packages are needed for this lab. If you want them anyway, install:

```
sudo apt install -y qemu-system-x86 libvirt-daemon-system libvirt-clients bridge-utils
```

Hardware acceleration only works if Linux exposes the KVM device. Check with:

```
ls -l /dev/kvm
```

If that file exists, add yourself to the kvm group and restart WSL:

```
sudo usermod -aG kvm $USER
# then, from Windows PowerShell:
wsl --shutdown
```

If /dev/kvm does not exist (common under WSL), skip the usermod step entirely and let QEMU fall back to software emulation with -accel tcg.

1.5 Mac users

Mac does not use WSL, so the steps below replace Steps 1.1–1.4 entirely. They work the same way on Apple Silicon (M1/M2/M3/M4) and Intel Macs. Once you reach "Launch QEMU" you re-join the same instructions everyone else follows in Step 2.

1.5.1 Install Xcode command line tools and Homebrew

Open Terminal and run:

```
xcode-select --install
```

Accept the prompt to install the developer tools, then install Homebrew if you do not already have it:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow the on-screen instructions at the end of the Homebrew installer to add brew to your shell PATH (it will print the exact lines to add to your .zprofile), then restart Terminal.

1.5.2 Install QEMU

```
brew install qemu wget
```

Verify:

```
qemu-system-aarch64 --version
```

Note: Homebrew's qemu package already includes qemu-system-aarch64, qemu-img, and every other QEMU binary you need — there is no separate "qemu-system-arm" package to install as there is on Ubuntu/WSL.

1.5.3 Get the Raspberry Pi image

```
cd ~
wget https://downloads.raspberrypi.com/raspios_arm64/images/raspios_arm64-2023-05-03/2023-05-03-raspios-bullseye-arm64.img.xz
xz -d 2023-05-03-raspios-bullseye-arm64.img.xz
```

1.5.4 Mount the boot partition (macOS-specific step)

macOS does not support Linux's "mount -o loop,offset=..." syntax, so the mount step in Step 2.2 of this handout does not apply on a Mac. Instead, use hdiutil, which reads the partition table and mounts each partition automatically:

```
hdiutil attach -imagekey diskimage-class=CRawDiskImage ./2023-05-03-raspios-bullseye-arm64.img
```

The output lists two volumes, for example:

```
/dev/disk4          FDisk_partition_scheme
```

/dev/disk4s1	Windows_FAT_32	/Volumes/boot
/dev/disk4s2	Linux	

The FAT32 line is the boot partition, already mounted at a path such as /Volumes/boot (the exact name may vary). Copy the kernel and device tree from it:

```
cp /Volumes/boot/bcm2710-rpi-3-b-plus.dtb ~
cp /Volumes/boot/kernel8.img ~
```

Then detach the image before continuing:

```
hdiutil detach /dev/disk4
```

Note: Use the actual /dev/diskN value hdiutil printed for your image, which can differ from the disk4 example above.

1.5.5 Resize the image

Same as Step 2.3 for everyone else:

```
qemu-img resize ./2023-05-03-raspbios-bullseye-arm64.img 32G
```

1.5.6 Enable SSH

Generate the password hash:

```
openssl passwd -6
```

Re-attach the image to reach the boot partition again:

```
hdiutil attach -imagekey diskimage-class=CRawDiskImage ./2023-05-03-raspbios-bullseye-arm64.img
```

Then, using the /Volumes/boot path shown this time:

```
echo 'pi:<your_hash_here>' | sudo tee /Volumes/boot/userconf
sudo touch /Volumes/boot/ssh
hdiutil detach /dev/disk4 # use the disk number hdiutil reported
```

1.5.7 Launch QEMU

From here, use the exact same qemu-system-aarch64 command shown in Step 2.5 of this handout — it is identical on macOS:

```
qemu-system-aarch64 \
-m machine raspi3b -cpu cortex-a72 -nographic \
-dtb bcm2710-rpi-3-b-plus.dtb \
-m 1G -smp 4 \
-kernel kernel8.img \
-sd 2023-05-03-raspbios-bullseye-arm64.img \
-append "rw earlyprintk loglevel=8 console=ttyAMA0,115200 dwc_otg.lpm_enable=0 \
root=/dev/mmcblk0p2 rootdelay=1" \
-device usb-net,netdev=net0 -netdev user,id=net0,hostfwd=tcp::5010-:22
```

SSH in from another Terminal window:

```
ssh -p 5010 pi@localhost
```

Note: The raspi3b machine type emulates ARM peripherals in software (TCG) regardless of host CPU, so even on an Apple Silicon Mac this does not use Apple's Hypervisor.framework (HVF) acceleration. Boot times are similar to what Windows/WSL and Linux users see — expect several minutes, not seconds.

Continue with Step 2.6 (Expand the Filesystem Inside the Emulated Pi) exactly as written for all platforms.

Step 2. Install Raspberry Pi OS on QEMU

This step is based on this page for installing Raspberry Pi on QEMU: [Emulating a Raspberry Pi in QEMU](#)

However, we need more storage in the emulated Raspberry Pi, so please follow the updated instructions below.

1. Get the Raspberry Pi image

QEMU was already installed in Step 1.3. If you skipped it, install it now — note that the binary is called `qemu-system-aarch64`, but the package that provides it is `qemu-system-arm`:

```
sudo apt install -y qemu-system-arm
cd ~
wget https://downloads.raspberrypi.com/raspios_arm64/images/raspios_arm64-2023-05-03/2023-05-03-raspios-bullseye-arm64.img.xz
xz -d 2023-05-03-raspios-bullseye-arm64.img.xz
```

Note: If this dated build has since been removed from the archive, browse https://downloads.raspberrypi.com/raspios_arm64/images/ to find an available bullseye-arm64 image and substitute its filename in the `wget` command and in the `mount/cp` commands below.

2. Inspect and mount the boot partition

```
fdisk -l ./2023-05-03-raspios-bullseye-arm64.img
# Note the boot partition offset (should be 8192 * 512 = 4194304)
sudo mkdir /mnt/image
sudo mount -o loop,offset=4194304 ./2023-05-03-raspios-bullseye-arm64.img /mnt/image/
```

Extract the kernel and device tree:

```
cp /mnt/image/bcm2710-rpi-3-b-plus.dtb ~
cp /mnt/image/kernel8.img ~
```

3. Resize the image to 32 GB (or your desired size)

Instead of resizing to the next power of two, pick a larger number directly. 32 GB:

```
qemu-img resize ./2023-05-03-raspios-bullseye-arm64.img 32G
```

4. Enable SSH

Create a hashed password and add `userconf + ssh`. In the `echo` command, replace `<your_hash_here>` with what you got from running `"openssl passwd -6"`. Copy this command into a text file and edit it there before pasting it back.

```
openssl passwd -6
# type your password twice; copy the hash
echo 'pi:<your_hash_here>' | sudo tee /mnt/image/userconf
sudo touch /mnt/image/ssh
```

Unmount:

```
sudo umount /mnt/image
```

5. Launch QEMU with the expanded disk

```
qemu-system-aarch64 \
-machine raspi3b -cpu cortex-a72 -nographic \
-dtb bcm2710-rpi-3-b-plus.dtb \
-m 1G -smp 4 \
-kernel kernel8.img \
-sd 2023-05-03-raspios-bullseye-arm64.img \
```

```
-append "rw earlyprintk loglevel=8 console=ttyAMA0,115200 dwc_otg.lpm_enable=0 \
root=/dev/mmcblk0p2 rootdelay=1" \
-device usb-net,netdev=net0 -netdev user,id=net0,hostfwd=tcp::5010-:22
```

To log into the Pi, enter pi as user name, and the password you sent previously.

You can log into the Pi within the current window, or you can SSH in from another WSL window:

```
ssh -p 5010 pi@localhost
```

Save the above commands in a text file (for example start-pi.sh) for starting up the simulator in the future.

Note: Do not add `-enable-kvm` or `-accel kvm` to this command. The guest is ARM and your PC is x86, so QEMU must use software translation. Booting to the login prompt may take several minutes.

6. Expand the filesystem inside the emulated Pi

Once inside the Pi:

```
sudo raspi-config
```

- Go to Advanced Options -> Expand Filesystem
- Reboot

Alternatively (manual method, usually not needed):

```
sudo fdisk /dev/mmcblk0
# delete partition 2 and recreate it to extend to the full disk
sudo reboot
sudo resize2fs /dev/mmcblk0p2
```

7. Verify the new size

```
df -h
```

You should now see nearly 32 GB available in the root folder /.

This modified workflow only changes Step 3 (resize) and Step 6 (expand FS); everything else is identical to the original blog post's process.

Step 3. Enable 32-bit ARMv7 assembly on Raspberry Pi 3/4

Follow the next steps for running 32-bit ARMv7 assembly programs on a Raspberry Pi 3/4 running Raspberry Pi OS 64-bit.

1. Verify hardware and OS architecture

```
uname -m          # should show aarch64
lscpu | grep -i model
```

Raspberry Pi 3 and 4 (ARMv8-A) can run ARMv7 binaries if the kernel supports AArch32.

2. Enable 32-bit (armhf) support in Raspberry Pi OS 64-bit

By default the 64-bit OS is multi-arch capable. You need the 32-bit runtime:

```
sudo dpkg --add-architecture armhf
sudo apt update
sudo apt install libc6:armhf libstdc++6:armhf
```

You also need the 32-bit assembler and linker used in the next section:

```
sudo apt install binutils-arm-linux-gnueabi
```

If you want to compile C programs for ARMv7 as well (not required for this lab):

```
sudo apt install gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
```

Note: Earlier versions of this handout listed binutils-arm-linux-gnueabi as optional. It is not — without it the arm-linux-gnueabi-as and arm-linux-gnueabi-ld commands below do not exist.

3. Generating an ARMv7 binary

Save your ARMv7 assembly program, e.g. hello.s, using a text editor (see the vi tutorial). The program is from this page: <https://embeddedjourneys.com/blog/hello-world-arm-assembly-raspberry-pi/>

(Note: this program does not work in CPUlator, since CPUlator does not implement Linux system calls via SWI 0.)

```
.global _start
_start:
    MOV    R0, #1
    LDR    R1, =msg
    MOV    R2, #12
    MOV    R7, #4
    SWI    0
    MOV    R7, #1
    SWI    0
msg:
    .asciz "Hello, ARM!\n"
```

Note: The directive is .global with a leading dot. Writing global _start without the dot is a common source of "undefined reference" errors at link time.

Then assemble and link with the 32-bit toolchain:

```
arm-linux-gnueabi-as -o hello.o hello.s
arm-linux-gnueabi-ld -o hello hello.o
```

You will notice the output binary is ELF32 for ARM, which runs in AArch32 compatibility mode. You can create command aliases to reduce typing, c.f. *How to Create Aliases (Shortcuts) for Common Commands in Linux*. For example, if you are running the bash shell, edit .bashrc:

```
vi ~/.bashrc
```

and add these lines:

```
alias as32='arm-linux-gnueabi-as'
alias ld32='arm-linux-gnueabi-ld'
alias gcc32='arm-linux-gnueabi-gcc'
```

Note: Use plain ASCII straight quotes in .bashrc. Curly quotes copied from a PDF or word processor will produce a syntax error.

Check with:

```
file hello
# ./hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, not
stripped
```

4. Run the ARMv7 program directly

```
./hello
```

The Pi's 64-bit kernel automatically switches into AArch32 user mode for execution.

5. Debug / inspect (optional)

You can disassemble or inspect with:

```
arm-linux-gnueabihf-objdump -d hello
```

Step 4. Lab Report

Write a short report that records your experience in going through these steps. What problems did you encounter, and how did you resolve them?

Changes in this revision

- Step 1: qemu-kvm replaced with qemu-system-arm and qemu-utils. On current Ubuntu releases qemu-kvm is a virtual package and the old command fails with "has no installation candidate".
- Step 1: libvirt, bridge-utils and the kvm group membership moved to an optional section, since cross-architecture emulation of the Pi cannot use KVM.
- Step 1: Hyper-V marked optional, with a note for Windows 11 Home users.
- Step 2.1: package name corrected from qemu-system-aarch64 (not a package) to qemu-system-arm, which provides the qemu-system-aarch64 binary.
- Step 2: duplicated "8. Verify the New Size" section removed.
- Step 3.2: binutils-arm-linux-gnueabihf moved from optional to required.
- Step 3.3: .global directive corrected (leading dot) and the listing indented.
- Step 2.1: download URL updated from downloads.raspberrypi.org (404) to downloads.raspberrypi.com, the current Raspberry Pi download domain.
- Step 1.5 (Mac users): replaced the three external links with full step-by-step commands (Homebrew, brew install qemu, hdiutil-based partition mounting in place of Linux's mount -o loop,offset=..., and the exact QEMU launch command), since macOS mounting and package management differ enough from WSL/Linux to need their own instructions.